

Отчёт по докладу Модель сегрегации Шеллинга

Дисциплина: Математическое моделирование

Боровиков Даниил Александрович НПИбд-01-22

Содержание

1	Цель работы	5
2	Введение	6
3	Модель сегрегации Шеллинга	7
4	Python-реализация модели Шеллинга	9
5	Выводы	19
	Список литературы	20

Список иллюстраций

4.1	Модель Шеллинга начальное состояние	15
4.2	Модель Шеллинга порог сходимости 30%	16
4.3	Модель Шеллинга порог сходимости 50%	16
4.4	Модель Шеллинга порог сходимости 70%	17
4.5	График зависимости среднего значения сходства для разных порогов сходства	18

Список таблиц

1 Цель работы

Решить проблему математической модели сегрегации Шеллинга.

2 Введение

В информатике агентные модели используются для оценки влияния автономных агентов (то есть отдельных лиц, групп или объектов) на систему в целом. Это очень мощные аналитические инструменты, которые можно использовать в ситуациях, когда проведение экспериментов нецелесообразно или очень дорого. Эти модели имеют широкий спектр применения в социальных науках, информатике, экономике и бизнесе.

В данном докладе мы познакомимся с возможностями агентных моделей, которые используются для понимания сложных явлений. Для этого мы воспользуемся Python и моделью Шеллинга.

3 Модель сегрегации Шеллинга

Модель сегрегации Шеллинга — это агентная модель, разработанная экономистом Томасом Шеллингом. Модель Шеллинга не включает внешние факторы, которые оказывают давление на агентов с целью сегрегации, но работа Шеллинга демонстрирует, что наличие людей с “умеренным” внутригрупповым предпочтением по отношению к своей собственной группе все еще может привести к сильно сегрегированному обществу через фактическую сегрегацию. [1].

В агентно-ориентированных моделях есть три параметра: 1) агенты, 2) поведение (правила) и 3) показатели на агрегированном уровне. В модели Шеллинга агентами являются люди, живущие в городе, поведением — переезд в другой район в зависимости от коэффициента сходства, а показателями на агрегированном уровне — коэффициент сходства.

Пусть n — количество рас, проживающих в городе. Мы обозначаем каждую расу уникальным цветом, а город — сеткой, в которой каждая ячейка обозначает дом. Дом может быть пустым или полным. В полном доме может жить только один человек. Если дом пуст, мы закрашиваем его белым цветом. Если дом полон, мы закрашиваем его цветом человека, живущего в нём. Мы определяем соседей человека как людей, живущих в соседних домах (выше, ниже, слева, справа, выше-слева, выше-справа, ниже-слева, ниже-справа).

Цель Шеллинга состояла в том, чтобы проверить, что произойдёт, если люди будут отдавать небольшое предпочтение соседям своего цвета кожи (или даже предпочтут некоторое разнообразие). Если соотношение соседей одной расы превышает определённый порог (который мы называем порогом сходства), мы

говорим, что человек удовлетворён. Если нет, мы говорим, что человек неудовлетворён.

Моделирование Шеллинга происходит следующим образом. Мы начинаем со случайного размещения людей в городе и оставляем некоторые дома пустыми. Для каждого человека, живущего в городе, мы проверяем, доволен ли он. Если доволен, мы ничего не делаем. Если нет, мы переселяем человека в пустой дом. Мы запускаем это моделирование на несколько итераций и проверяем итоговое распределение людей. [2].

4 Python-реализация модели Шеллинга

```
import matplotlib.pyplot as plt
import itertools
import random
import copy

class Schelling:
    def __init__(self, width, height, empty_ratio, similarity_threshold, n_iterations, races=2):
        # Инициализация параметров модели
        self.width = width # Ширина сетки
        self.height = height # Высота сетки
        self.races = races # Количество групп (рас)
        self.empty_ratio = empty_ratio # Доля пустых домов
        self.similarity_threshold = similarity_threshold # Порог схожести для удовлетворённости
        self.n_iterations = n_iterations # Максимальное количество итераций
        self.empty_houses = [] # Список пустых домов
        self.agents = {} # Словарь агентов (координаты: группа)

    def populate(self):
        # Создание списка всех домов и их случайное перемешивание
        self.all_houses = list(itertools.product(range(self.width), range(self.height)))
```

```

random.shuffle(self.all_houses)

# Определение количества пустых домов
self.n_empty = int(self.empty_ratio * len(self.all_houses))
self.empty_houses = self.all_houses[:self.n_empty]

# Распределение оставшихся домов между группами
self.remaining_houses = self.all_houses[self.n_empty:]
houses_by_race = [self.remaining_houses[i::self.races] for i in range(self.races)]
for i in range(self.races):
    # Добавление агентов каждой группы в словарь
    self.agents.update({house: i + 1 for house in houses_by_race[i]})

def is_unsatisfied(self, x, y):
    # Проверка, удовлетворён ли агент на позиции (x, y)
    race = self.agents[(x, y)] # Группа агента
    count_similar = 0 # Количество соседей той же группы
    count_different = 0 # Количество соседей другой группы

    # Список координат всех 8 соседей
    neighbors = [
        (x - 1, y - 1), (x, y - 1), (x + 1, y - 1),
        (x - 1, y),          (x + 1, y),
        (x - 1, y + 1), (x, y + 1), (x + 1, y + 1)
    ]

    for nx, ny in neighbors:
        # Проверка, находится ли сосед в пределах сетки и не является ли дом пустым
        if (0 <= nx < self.width and 0 <= ny < self.height and

```

```

        (nx, ny) not in self.empty_houses and (nx, ny) in self.agents):
    if self.agents[(nx, ny)] == race:
        count_similar += 1
    else:
        count_different += 1

# Если соседей нет, агент удовлетворён
if count_similar + count_different == 0:
    return False

# Агент не удовлетворён, если доля похожих соседей ниже порога
return count_similar / (count_similar + count_different) < self.similarity_threshold

def update(self):
    # Выполнение симуляции для заданного числа итераций
    for i in range(self.n_iterations):
        self.old_agents = copy.deepcopy(self.agents) # Копия текущего состояния
        n_changes = 0 # Счётчик перемещений
        for agent in self.old_agents:
            if self.is_unsatisfied(agent[0], agent[1]):
                agent_race = self.agents[agent] # Группа агента
                empty_house = random.choice(self.empty_houses) # Выбор пустого дома
                self.agents[empty_house] = agent_race # Перемещение агента
                del self.agents[agent] # Удаление старой позиции
                self.empty_houses.remove(empty_house) # Обновление списка пустых домов
                self.empty_houses.append(agent)
                n_changes += 1
        print(f"Итерация {i + 1}: {n_changes} агентов перемещено")
        if n_changes == 0: # Прерывание, если перемещений нет
            break

```

```

def plot(self, title, file_name):
    # Создание визуализации текущего состояния сетки
    fig, ax = plt.subplots()
    # Цвета для групп (до 7 групп)
    agent_colors = {1: 'b', 2: 'r', 3: 'g', 4: 'c', 5: 'm', 6: 'y', 7: 'k'}
    for agent in self.agents:
        # Отрисовка агента с соответствующим цветом
        ax.scatter(agent[0] + 0.5, agent[1] + 0.5, color=agent_colors[self.agents[agent]])

    ax.set_title(title, fontsize=10, fontweight='bold')
    ax.set_xlim([0, self.width])
    ax.set_ylim([0, self.height])
    ax.set_xticks([])
    ax.set_yticks([])
    plt.savefig(file_name) # Сохранение графика
    plt.close(fig) # Закрытие фигуры для предотвращения утечек памяти

def calculate_similarity(self):
    # Вычисление среднего коэффициента схожести для всех агентов
    similarity = []
    for agent in self.agents:
        count_similar = 0 # Количество соседей той же группы
        count_different = 0 # Количество соседей другой группы
        x, y = agent
        race = self.agents[(x, y)]

        # Список координат всех 8 соседей
        neighbors = [

```

```

(x - 1, y - 1), (x, y - 1), (x + 1, y - 1),
(x - 1, y),          (x + 1, y),
(x - 1, y + 1), (x, y + 1), (x + 1, y + 1)
]

```

```

for nx, ny in neighbors:

```

```

    # Проверка, находится ли сосед в пределах сетки и не является ли дом пустым

```

```

    if (0 <= nx < self.width and 0 <= ny < self.height and

```

```

        (nx, ny) not in self.empty_houses and (nx, ny) in self.agents):

```

```

        if self.agents[(nx, ny)] == race:

```

```

            count_similar += 1

```

```

        else:

```

```

            count_different += 1

```

```

# Вычисление коэффициента схожести

```

```

if count_similar + count_different > 0:

```

```

    similarity.append(float(count_similar) / (count_similar + count_different))

```

```

else:

```

```

    similarity.append(0) # Если соседей нет, коэффициент 0

```

```

# Возврат среднего коэффициента схожести

```

```

return sum(similarity) / len(similarity) if similarity else 0

```

```

# Инициализация трёх симуляций с разными порогами схожести

```

```

schelling_1 = Schelling(50, 50, 0.3, 0.3, 500, 2)

```

```

schelling_1.populate()

```

```

schelling_2 = Schelling(50, 50, 0.3, 0.5, 500, 2)

```

```

schelling_2.populate()

```

```

schelling_3 = Schelling(50, 50, 0.3, 0.7, 500, 2)
schelling_3.populate()

# Сохранение начального состояния
schelling_1.plot('Модель Шеллинга с 2 группами: Начальное состояние', 'schelling_2_initial.png')

# Запуск симуляций
schelling_1.update()
schelling_2.update()
schelling_3.update()

# Сохранение финальных состояний
schelling_1.plot('Модель Шеллинга с 2 группами: Финальное состояние, порог 30%', 'schelling_2_30_final.png')
schelling_2.plot('Модель Шеллинга с 2 группами: Финальное состояние, порог 50%', 'schelling_2_50_final.png')
schelling_3.plot('Модель Шеллинга с 2 группами: Финальное состояние, порог 70%', 'schelling_2_70_final.png')

# Анализ зависимости порога схожести от среднего коэффициента схожести
similarity_threshold_ratio = {}
for i in [0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7]:
    schelling = Schelling(50, 50, 0.3, i, 500, 2)
    schelling.populate()
    schelling.update()
    similarity_threshold_ratio[i] = schelling.calculate_similarity()

# Построение графика зависимости
fig, ax = plt.subplots()
plt.plot(list(similarity_threshold_ratio.keys()), list(similarity_threshold_ratio.values()), 'ro')
ax.set_title('Порог схожести vs Средний коэффициент схожести', fontsize=15, fontweight='bold')

```

```
ax.set_xlim([0, 1])  
ax.set_ylim([0, 1.1])  
ax.set_xlabel("Порог схожести")  
ax.set_ylabel("Средний коэффициент схожести")  
plt.savefig('schelling_segregation_measure.png')  
plt.close(fig) # Закрытие фигуры
```

Ниже представлен результат работы нашей программы (рис. 4.1) (рис. 4.2).
(рис. 4.3). (рис. 4.4).

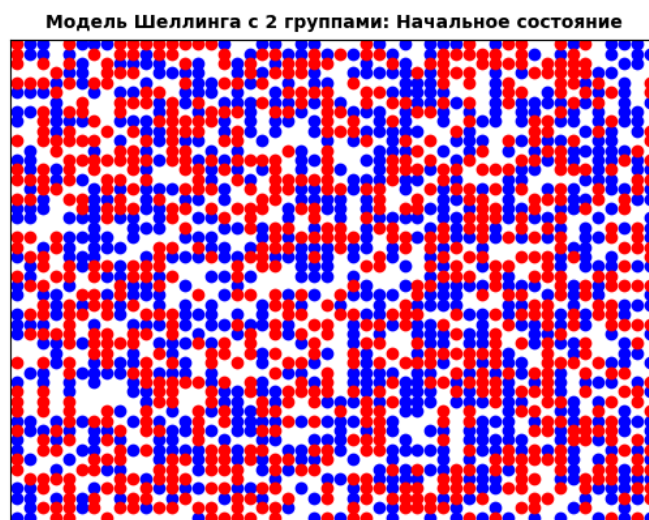


Рис. 4.1: Модель Шеллинга начальное состояние

Модель Шеллинга с 2 группами: Финальное состояние, порог 30%

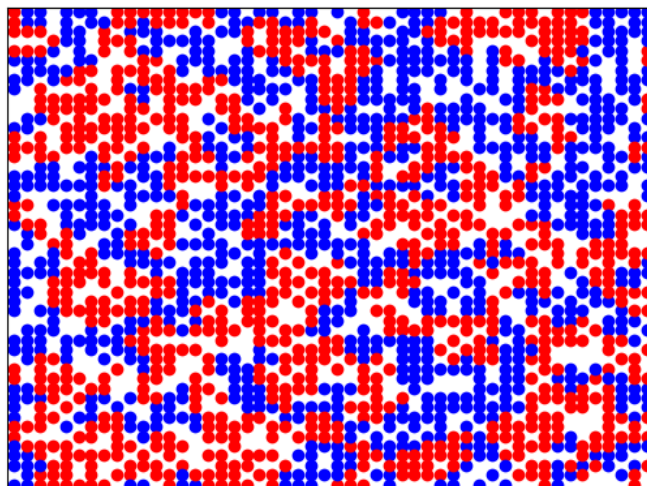


Рис. 4.2: Модель Шеллинга порог сходимости 30%

Модель Шеллинга с 2 группами: Финальное состояние, порог 50%

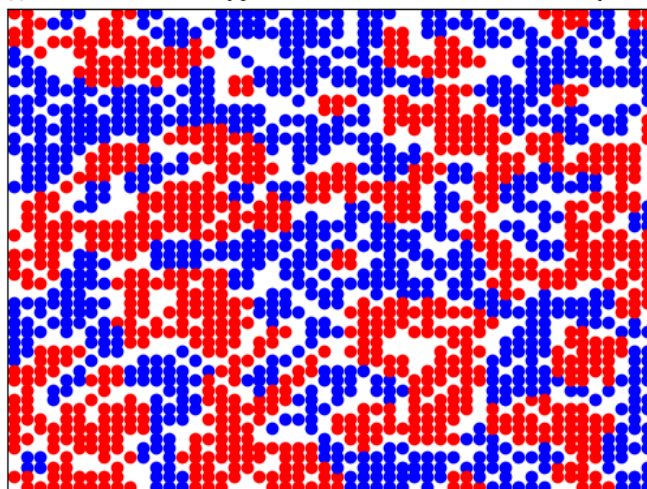


Рис. 4.3: Модель Шеллинга порог сходимости 50%

Модель Шеллинга с 2 группами: Финальное состояние, порог 70%

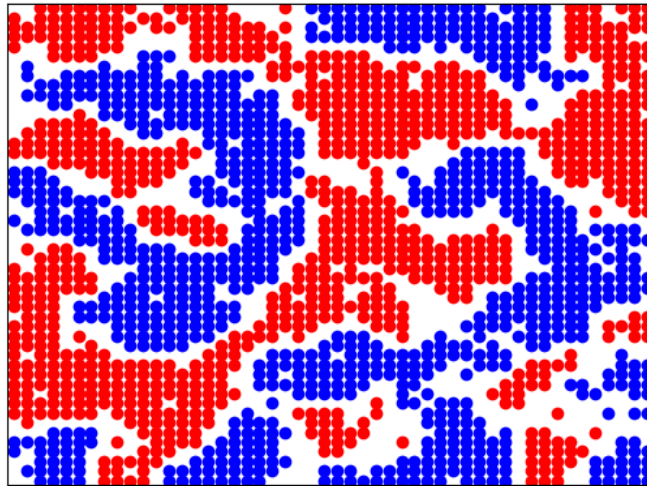


Рис. 4.4: Модель Шеллинга порог сходимости 70%

Глядя на приведённые выше графики, мы видим, что чем выше порог сходства, тем более сегрегированным становится город. Кроме того, мы видим, что даже при небольшом пороге сходства мы получаем сегрегированные города. Другими словами, даже если люди очень терпимы (то есть имеют небольшой порог сходства), мы всё равно получаем сегрегацию.

Теперь измерим сегрегацию, для этого мы вычислим среднее значение коэффициента сходства для разных порогов сходства и построим график зависимости между порогом сходства и коэффициентом сходства.

(рис. 4.5).

Порог схожести vs Средний коэффициент схожести

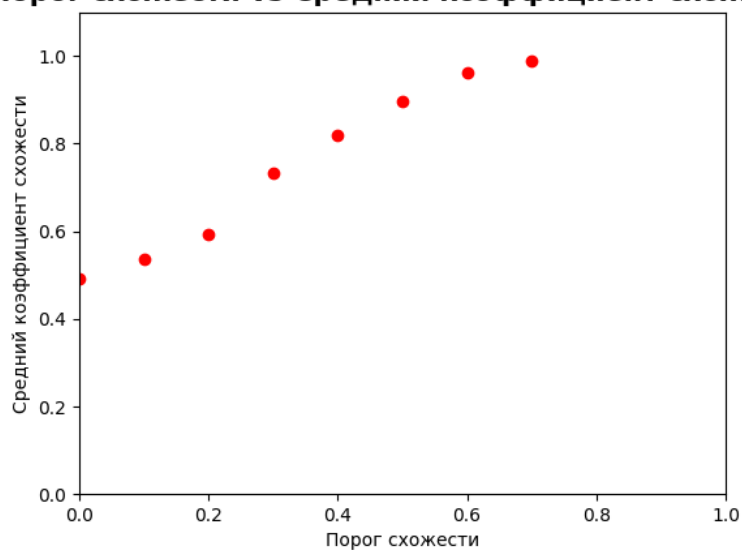


Рис. 4.5: График зависимости среднего значения сходства для разных порогов сходства

Из приведённого выше графика видно, что при очень низких пороговых значениях сходства мы получаем очень высокую степень сегрегации (определяемую средним коэффициентом сходства). Например, при пороговом значении сходства 0,3 мы получили средний коэффициент сходства 0,75.

5 Выводы

Мы рассмотрели один пример агентных моделей под названием «Модель сегрегации Шеллинга» и реализовали её на Python. Эта очень простая модель помогла нам понять очень сложное явление, а именно сегрегацию в многонациональных городах. Мы смогли показать, что очень высокий уровень сегрегации в этих городах не обязательно приводит к нетерпимости на индивидуальном уровне.

Список литературы

1. model_of_segregation [Электронный ресурс]. URL: https://en.wikipedia.org/wiki/Schelling%27s_model_of_segregation.
2. Таненбаум Э., Бос Х. Современные операционные системы. 4-е изд. СПб.: Питер, 2015. 1120 с.